Overview of Logistic Regression

What is Logistic Regression loss?

$$\min_{x \in R^n} l(x) = \frac{1}{m} \sum_{i=1}^{m} \ln(1 + \exp(-b_i a_i^T x)) + \lambda ||x||^2$$

- 其中， $\boldsymbol{a_i \in R^n, b_i \in \{-1, +1\}, i = 1, 2, \ldots, m}$ 是给定的数据，$\boldsymbol{\lambda > 0}$ 是给定的正数。

Application

- 应用于二分类问题。

The Mathematics Behind Logistic Regression

$$\min_{x\in R^n} l(x) = \frac{1}{m}\sum_{i=1}^{m} \ln(1+\exp(-b_i a_i^T x)) + \lambda||x||^2$$

- 计算函数值:

➢ 令 $A = \begin{pmatrix} a_1^T \\ a_2^T \\ \cdots \\ a_m^T \end{pmatrix}$, $b = \begin{pmatrix} b_1 \\ b_2 \\ \cdots \\ b_m \end{pmatrix}$

➢ 先计算Ax，再计算 $b.*(Ax)$，得到$\begin{pmatrix} b_1 a_1^T x \\ b_2 a_2^T x \\ \cdots \\ b_m a_m^T x \end{pmatrix}$

➢ MATLAB中，exp 函数变量为向量时，表示对每个分量分别求指数:

➢ 故

$$\frac{1}{m}\sum_{i=1}^{m} \ln(1+\exp(-b_i a_i^T x)) = \frac{1}{m} sum(\log(1+\exp\begin{pmatrix} -b_1 a_1^T x \\ -b_2 a_2^T x \\ \cdots \\ -b_m a_m^T x \end{pmatrix}))$$

- 计算梯度

令 $c = \exp\begin{pmatrix} -b_1 a_1^T x \\ -b_2 a_2^T x \\ \cdots \\ -b_m a_m^T x \end{pmatrix}$

$$\nabla l(x) = \frac{1}{m} \cdot \sum_i^m \frac{-b_i a_i \exp(-b_i a_i^T x)}{1+\exp(-b_i a_i^T x)} + 2\lambda x$$

$$= \frac{1}{m} \cdot \sum_i^m \frac{-b_i a_i}{1+\exp(b_i a_i^T x)} + 2\lambda x$$

$$= \frac{1}{m} \cdot \sum_i^m \left(\frac{b_i}{1+\exp(-b_i a_i^T x)} - b_i \right) a_i + 2\lambda x$$

$$= \frac{1}{m} A^T (b. \div (1 + c) - b) + 2\lambda x$$

向量化计算，不写for 循环

不重复计算，保存需要的变量

Backtracking linesearch

1. 选取初始步长$\alpha_0, \gamma \in (0,1)$和$c \in (0,1)$

2. 找到最小的正数$t \geq 0$,使得
$$f(x_k + \alpha_0 \gamma^t d_k) \leq f(x_k) + c\alpha_0 \gamma^t \nabla f(x_k)^T d_k$$

3. 令$\alpha_k = \alpha_0 \gamma^t$,更新$x_{k+1} = x_k + \alpha_k d_k$

手动调整参数γ和c, 估计α_0
设置步长搜索下界

代码规范要求

- 注释:
  - 通过注释解释每块代码的意义。分别从做什么、为什么、怎么做 来进行注释。结构体和函数一定要写注释，而且要写得尽可能全面、详细，而函数内部的注释要相对少一些，一般都是靠好的命名、提炼函数、解释性变量、总结性注释来提高代码可读性。
- 可读性:
  - 使用规范的文件名、函数名、变量名增加代码的可读性
- 格式要求:
  - 注意缩进、对齐等；一行只做一件事。

代码结构和组织

模块化:

- 善于将代码中的模块进行抽象，能够方便我们的阅读(e.g.，objective function，gradient calculation，BFGS update).
- 函数职责需要单一，避免设计一个大而全的函数，避免函数或方法参数过多

命名:

- 关于命名长度，在能够表达含义的额情况下，命名当然是越短越好。
- 通用的缩写要统一，例如 var，obj
- 多个单词连在一起，可以采用驼峰命名：例如 FirstName
- 避免使用易混淆的字符：如O(大o）和0（数字0），l（小写L）和I大写（i）

提高效率的一些方法

- 向量化　　例：　勿使用for loop 计算可向量化的值
- 勿重复计算　例：梯度和函数值中重复出现的中间值

- 稀疏矩阵乘法 ：稀疏矩阵可以加速计算
- 一些优化代码的建议：https://ww2.mathworks.cn/help/matlab/matlab_prog/techniques-for-improving-performance.html

测试和调试

- 测试：可以针对局部模块进行简单测试
- Debugging Tips：善于应用Debugging和Profiling

牛顿法

- Hessian矩阵如下：

$$\nabla^2 l(x) = \frac{1}{m} \cdot \sum_{i=1}^{m} \frac{b_i^2 \exp\left(b_i a_i^T x\right)}{[1 + \exp\left(b_i a_i^T x\right)]^2} a_i a_i^T + 2\lambda I$$

- 可以估计梯度的Lipschitz常数为$\frac{1}{4m}||A||_2^2$

- 求解线性方程

$$\nabla^2 l(x) d = -\nabla l(x)$$

BFGS Algorithm

- **Algorithm Steps**: Break down the BFGS algorithm into clear, sequential steps.
- **Updating Rules**: Clearly implement the BFGS updating rules for the approximate Hessian matrix and the solution vector.
- **Convergence Criteria**: Set and explain the criteria for convergence to stop the algorithm.

作图

- 要求可辨别收敛速率：semilogy作图
- 图片清晰，文字、线条可以达到打印清晰可见

一些工程技巧

- 线搜索中，每步初始步长可以延续上一个步骤的步长。
- 初始迭代步时，可以非精确求解。

Project 3实验报告要求

总分10分

- 代码规范性2分：包括命名规范和函数模块化
- 正确性3分
- 实验报告5分：
  - 简要说明相关量的计算方法，和你采用的加速技巧。
  - 报告a9a的收敛速度图像，包括
    - 函数值和最优值的差的收敛速度
    - 梯度大小的收敛速度
    - 报告 线搜索参数和收敛速度的简要关系

Project1 线性规划要求

可以假设输入的线性规划必须是标准形式

总分10分

- 代码规范性2分（包括命名规范和函数模块化）+正确性4分：
  - 模块1： 检查A是否行慢秩，如果非满秩，移除多余的约束
  - 模块2： 初始化可行基解：大M法
  - 模块3：单纯形法的迭代
  - 正确性还包括避免退化解循环，需要验证并写入报告。
- 实验报告4分
  - 说明各个模块中，是如何处理的，可以结合讲义。
  - 随机生成测试案例，报告求解时间随着问题规模的变化图像。每个规模的时间，需要对随机的20个测试案例取均值，以及计算标准差。

数学规划求解器：国内外现状

- 1939年，苏联数学家和经济学家Leonid Kantorovich发明线性规划
- 1979年全球首个优化Charge 发布Lingo
- 1983年美国卡耐基梅隆Ashford创建XPRESS
- 1987年美国卡耐基梅隆Bixby创建CPLEX公司
- 2000年COIN-OR成立，开放源代码CLP和CBC
- 2005年德国ZIB发布了开源整数规划工具SCIP
- 2008年Cplex创始人Bixby离开创办GUROBI
- 2017年，上海交大创建一个开源数学规划工具LEAVES
- 2018年中科院推出CMIP

- GUROBI
- CPLEX
- XPRESS

美国与英国三大求解器巨头，累计三十年研发历史和95%以上市场。线性、整数、非线性模块功能齐全

- SAS：最大的商业统计软件（北卡）
- CVX：最著名的求解器建模平台（斯坦福）
- IPOPT：著名的非线性规划开源求解全局（卡耐基梅隆）
- Coin-OR：最好的开源线性规划（多组织维护）
- Baron：最好的非线性规划（多组织维护）
- NEOS：最大的优化求解免费平台
- SCIP：最好的整数规划
- CBC：美国
- SOPLEX：德国
- MOSEK：丹麦
- GLPK：俄罗斯

简单的前置处理

- A是否有全为0的行，此时b如何？
- 是否有简单的线性相关行 $[a_i\ b_i] = \lambda[a_j\ b_j]$
- A是否有全为0的列

模块1：检查A是否满秩：方法1

给定$Ax \le b$, 如何得到所有行都是线性无关的矩阵？

可以考虑这个问题：

$$
\begin{array}{lll}
f^* = & \text{maximize} & s^T x \\
 & \text{subject to} & Ax \le b \\
 & & s^T x \le t + 1.
\end{array}
$$

不等式$s^T x \le t$ 相对于 $Ax \le b$是冗余的，即s与A线性相关，当且仅当f^*小于等于t

该结论的提示：可以先从小的线性系统出发，例如$A_0 x \le b_0$, 这里A_0的行数很小，逐个增加新的不等式. 直到A的所有都行检查过。

模块1：检查A是否满秩：方法2

使用QR分解

- $\mathrm{A} \in \mathbf{R}^{\{\mathrm{m}\times\mathrm{n}\}}$，记$\mathrm{B} = A^T$
- 有如下分解：$\mathrm{B} = \mathrm{QR}$，其中Q$\in \mathbf{R}^{\{\mathrm{n}\times m\}}$ 是正交矩阵，R的对角元的绝对值从大到小排列。
- 如果 R的对角元都非零，则A满秩。
- 反之，R有对角元为零，说明可以删掉对应的Q的列,得到B′ 。
- 最后得到满秩矩阵 $\mathrm{A}' = (B')^T$
- 向量b删掉对应的行元素即可

模块1：检查A是否满秩：方法？

- **你有何种方法？**

可以通过检索文献，直接实现，说明大概是如何处理的即可。无需比前两种更优越。

通过如下代码检查你的函数：

- % Define a matrix A with linearly dependent rows
- A = [1 2 3 4; 1 1 3 4; 2 3 6 8];
-
- % Define a corresponding vector b
- b = [6; 12; 15];

下一页继续。。。

模块1：检查A是否满秩：方法？

```
% Display the original matrix and vector
disp('Original Matrix A:');
disp(A);
disp('Original Vector b:');
disp(b);

% Apply the makeFullRankQR function
[A_fullrank, b_fullrank] = makeFullRankQR(A, b);
% Display the modified matrix and vector
disp('Modified Full Rank Matrix A:');
disp(A_fullrank);
disp('Modified Vector b:');
```

模块2：初始可行基解

大M法

- 如何确定M的大小？
  - 太小：非等价于原始问题
  - 太大：容易出现数值误差
  - 动态更新？
- 是否会添加冗余的列？

或许两阶段法更合适？

模块3： 单纯形法的迭代

- 给定可行基解 x, 对应基坐标集合B和非基坐标N
- 计算最优判定条件
  - $r_{cost} = c_j - c_B^T B^{-1} A_j, j \in N$. 确定入基变量
  - 计算$\frac{x_{B_i}}{d_{B_i}}$确定出基变量
  - 避免求逆矩阵
- 采用Bland's rule避免陷入循环

Bland's rule

- 一直选取满足出基入基条件最小的下标
- 测试案列：讲义中的例子

Example 4.3 假设我们有如下初始单纯形表：

		x_1	x_2	x_3	x_4	x_5	x_6	x_7
	3	$-3/4$	20	$-1/2$	6	0	0	0
$x_5 =$	0	$1/4^*$	-8	-1	9	1	0	0
$x_6 =$	0	$1/2$	-12	$-1/2$	3	0	1	0
$x_7 =$	1	0	0	1	0	0	0	1

采用如下规则选取出基入基下标：

1. 对于非基向量，选取下标 j 使得减少量 $\bar{c}_j$ 最小
2. 对于基向量，出基下标，选取所有满足条件中下标最小的。